A MINI PROJECT REPORT ON

CREAMY BITES

ONLINE BAKERY E-COMMERCE PLATFORM WITH INTELLIGENT SALES PREDICTION, LOYALTY MANAGEMENT AND AI SHOPPING ASSISTANT

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS

Submitted by

AUDHIDYA SIBIN R.

[Register Number]

Under the guidance of

[Guide Name], [Designation]

DEPARTMENT OF COMPUTER APPLICATIONS

[NAME OF THE COLLEGE]

(Affiliated to [Name of the University])

CHENNAI, TAMIL NADU

[Month and Year of Submission]

[NAME OF THE COLLEGE]
DEPARTMENT OF COMPUTER APPLICATIONS

CERTIFICATE

This is to certify that the mini project report entitled **"CREAMY BITES – Online Bakery E-Commerce Platform"** submitted by **AUDHIDYA SIBIN R.** ([Register Number]) to [NAME OF THE COLLEGE], in partial fulfilment of the requirements for the award of the degree of **BACHELOR OF COMPUTER APPLICATIONS**, is a bonafide record of the work carried out by him under my supervision and guidance during the period of study. The work embodied in this report has not been submitted earlier for the award of any other degree or diploma.

GUIDE	**HEAD OF THE DEPARTMENT**
[Guide Name]	[Head of the Department]
[Designation]	Department Of Computer Applications

Submitted for the viva-voice examination held on .....

INTERNAL EXAMINER	**EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

I am immensely grateful to the Almighty for the strength, health and perseverance that enabled me to complete this mini project successfully.

I express my deep sense of gratitude to **[Guide Name]**, [Designation], Department Of Computer Applications, for the constant encouragement, valuable guidance and constructive criticism offered throughout the course of this project. I am also thankful to **[Head of the Department]**, Head of the Department, and to all the faculty members of the Department Of Computer Applications for their support and suggestions.

I extend my sincere thanks to my friends and to my family members for their motivation and cooperation, without which this work would not have been completed in time.

AUDHIDYA SIBIN R.

ABSTRACT

The bakery and confectionery sector is one of the fastest growing segments of Indian food retail, yet a large number of neighbourhood bakeries continue to operate with manual order books, paper bills and no digital record of their customers. **Creamy Bites** is a full-stack, web-based e-commerce platform developed to modernise exactly such a bakery. The system provides a responsive customer storefront where shoppers can browse a catalogue organised into seven categories, search for products, add items to a cart or a wishlist, apply coupons and place orders against a chosen delivery slot. Every protected operation is secured with JSON Web Token authentication, and each order is confirmed with a four-digit delivery OTP shared through a WhatsApp receipt link.

On the administrative side, the application delivers a complete dashboard for managing products, categories, orders, inventory and customers. A Point-of-Sale (POS) module supports in-store counter billing, including quick registration of walk-in customers, real-time stock deduction and the same loyalty engine that serves online orders. The loyalty programme credits five reward points for every one hundred rupees spent, and one point is redeemable for one rupee; repeat customers are automatically identified and rewarded with graded discounts of five, ten and fifteen percent.

To reduce the wastage of perishable goods, the system includes a demand prediction module that applies Exponentially Weighted Moving Average smoothing to twenty days of historical orders and then scales the result with day-of-week and Indian festival multipliers, yielding next-day revenue, order-count and baking-quantity recommendations together with a confidence score. An AI shopping assistant answers natural-language queries about products, prices, ingredients and delivery. The application is built with React 18 and Vite on the front end, Node.js with Express on the back end and MongoDB Atlas for persistence. This report documents the problem statement, objectives, system analysis, design, implementation, testing and results of the project.

TABLE OF CONTENTS

CERTIFICATE . . . . . **ii**

ACKNOWLEDGEMENT . . . . . **iii**

ABSTRACT . . . . . **iv**

INTRODUCTION **1**

1.1 INTRODUCTION 1

1.2 OVERVIEW OF THE PROPOSED SYSTEM 1

1.3 ORGANISATION OF THE REPORT 2

SYSTEM ANALYSIS **3**

2.1 INTRODUCTION 3

2.2 PROBLEM STATEMENT 3

2.3 OBJECTIVES OF THE PROJECT 4

2.4 SCOPE OF THE PROJECT 5

2.4.1 In Scope 5

2.4.2 Out of Scope 6

2.4.3 Limitations 6

SYSTEM REQUIREMENTS AND TECHNOLOGY STUDY **7**

3.1 HARDWARE REQUIREMENTS 7

3.2 SOFTWARE REQUIREMENTS 7

3.3 TECHNOLOGY STUDY 8

3.3.1 React 18 8

3.3.2 Vite 8

3.3.3 Node.js and Express 8

3.3.4 MongoDB and Mongoose 8

3.3.5 JSON Web Token 9

3.3.6 bcryptjs 9

3.3.7 Axios 9

3.3.8 Multer and Morgan 9

3.4 FUNCTIONAL REQUIREMENTS 9

3.5 NON-FUNCTIONAL REQUIREMENTS 10

SYSTEM DESIGN 12

4.1 INTRODUCTION 12

4.2 SYSTEM ARCHITECTURE 12

4.3 DATA FLOW DIAGRAMS 13

4.3.1 Level 0 Data Flow Diagram 13

4.3.2 Level 1 Data Flow Diagram 14

4.4 USE CASE DIAGRAM 15

4.5 SEQUENCE DIAGRAM 16

4.6 ACTIVITY DIAGRAM 17

4.7 DATABASE DESIGN 18

4.7.1 Entity Relationship Diagram 18

4.7.2 Summary of Collections 19

4.7.3 Schema of the User Collection 20

4.7.4 Schema of the Product Collection 20

4.7.5 Schema of the Order Collection 21

IMPLEMENTATION 22

5.1 INTRODUCTION 22

5.2 DEVELOPMENT ENVIRONMENT AND PROJECT STRUCTURE 22

5.3 MODULE DESCRIPTION 23

5.3.1 Authentication and Authorisation Module 23

5.3.2 Category and Product Catalogue Module 23

5.3.3 Search and AI Assistant Module 24

5.3.4 Cart Module 24

5.3.5 Wishlist Module 24

5.3.6 Address Book Module 24

5.3.7 Coupon and Discount Module 24

5.3.8 Order Placement and Tracking Module 24

5.3.9 Loyalty Points Module 25

5.3.10 Repeat Customer Module 25

5.3.11 Inventory Management Module 25

5.3.12 Analytics and Demand Prediction Module 25

5.3.13 Point-of-Sale Module 25

5.3.14 Administration Panel Module 26

5.4 FRONT-END IMPLEMENTATION 26

5.4.1 Component Organisation 26

5.4.2 State Management 26

5.4.3 Routing 26

5.5 BACK-END IMPLEMENTATION AND REST API 27

5.6 DEMAND PREDICTION ALGORITHM 28

5.6.1 Steps of the Algorithm 28

5.6.2 Demand Multipliers 29

5.7 LOYALTY AND REPEAT-CUSTOMER ENGINE 29

5.8 OUTPUT SCREENS 30

5.8.1 Administration and Counter Screens 34

TESTING 38

6.1 INTRODUCTION 38

6.2 OBJECTIVES OF TESTING 38

6.3 TESTING STRATEGY 38

6.3.1 Unit Testing 38

6.3.2 Integration Testing 38

6.3.3 System Testing 38

6.3.4 Validation Testing 39

6.3.5 User Acceptance Testing 39

6.4 TEST CASES 39

6.5 RESULTS OF TESTING 40

RESULTS AND DISCUSSION 42

7.1 INTRODUCTION 42

7.2 ACHIEVEMENT OF OBJECTIVES 42

7.3 OBSERVATIONS 42

7.4 DISCUSSION 43

CONCLUSION AND FUTURE ENHANCEMENTS 44

8.1 CONCLUSION 44

8.2 FUTURE ENHANCEMENTS 44

REFERENCES 46

LIST OF FIGURES

Fig. 4.1 Three-tier architecture of the Creamy Bites platform 13

Fig. 4.2 Level 0 data flow diagram (context diagram) 14

Fig. 4.3 Level 1 data flow diagram 15

Fig. 4.4 Use case diagram of the proposed system 16

Fig. 4.5 Sequence diagram for placing a secured online order 17

Fig. 4.6 Activity diagram for the Point-of-Sale billing flow 18

Fig. 4.7 Entity relationship diagram of the Creamy Bites database 19

Fig. 5.1 Home Page of the Creamy Bites store front 30

Fig. 5.2 Product Listing Page 30

Fig. 5.3 Product Details Page 31

Fig. 5.4 Shopping Cart Page 31

Fig. 5.5 Checkout Page 31

Fig. 5.6 Login Page 32

Fig. 5.7 Registration Page 32

Fig. 5.8 Order Confirmation Screen 32

Fig. 5.9 My Orders Page 33

Fig. 5.10 Wishlist Page 33

Fig. 5.11 User Profile Page 33

Fig. 5.12 AI Shopping Assistant 34

Fig. 5.13 Administration Dashboard 34

Fig. 5.14 Product Management Screen 34

Fig. 5.15 Order Management Screen 35

Fig. 5.16 Inventory Management Screen 35

Fig. 5.17 Customer Management Screen 35

Fig. 5.18 New Customers Report 36

Fig. 5.19 Repeat Customers Report 36

Fig. 5.20 Loyalty Management Screen 36

Fig. 5.21 Sales Analytics Screen 37

Fig. 5.22 Demand Prediction Screen 37

Fig. 5.23 Point-of-Sale Counter Billing Screen 37

LIST OF TABLES

Table 3.1 Hardware requirements 7

Table 3.2 Software requirements 8

Table 4.1 Summary of MongoDB collections 19

Table 4.2 Schema of the users collection 20

Table 4.3 Schema of the products collection 20

Table 4.4 Schema of the orders collection 21

Table 5.1 REST API endpoints of the application 28

Table 5.2 Demand multipliers used by the prediction module 29

Table 5.3 Repeat-customer discount tiers 30

Table 6.1 Test cases and their results 40

Table 7.1 Objectives and their achievement 42

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

Commerce on the Internet has changed the way people buy everyday goods. What began as a convenient way to purchase books and electronics now covers almost every category of retail, including fresh food. Customers today expect to see a product, compare its price and pack size, read what other buyers think about it, place an order from wherever they are and receive a confirmation without having to make a telephone call. Businesses that cannot offer this experience slowly lose ground to competitors that can.

The bakery trade has its own special difficulties. Sweets, cakes, breads and savouries are perishable; most items sell for only one or two days before they lose freshness. A bakery therefore has to balance two opposite risks: producing too little and turning customers away during a festival rush, or producing too much and writing off unsold stock at the end of the day. Both risks can be reduced if the owner knows, with reasonable confidence, how much of each item the next day is likely to require.

A second difficulty is the absence of customer information. In a traditional bakery an order is written into a paper book or remembered by the person at the counter, so there is no reliable record of who bought what, how often, or how much they spent. Without that record the owner cannot recognise loyal customers, cannot reward them and cannot tell them about an offer that would suit them. The proposed system addresses both of these problems by combining an online store with a sales prediction engine and a loyalty programme.

1.2 OVERVIEW OF THE PROPOSED SYSTEM

Creamy Bites is a three-tier web application developed for a bakery operating in Chennai. The presentation tier is a single-page application built with React 18 and bundled by Vite; it renders a customer storefront, an administrative dashboard and a counter billing screen. The application tier is a RESTful API built with Node.js and Express, organised into controllers for authentication, products, categories, cart, orders, wishlist, addresses, coupons, reviews, administration and point-of-sale billing. The data tier is a MongoDB Atlas cluster accessed through the Mongoose object data modelling library.

Shoppers can register, log in, browse a catalogue divided into seven categories, search, filter by price and rating, add items to a cart or a wishlist, save multiple delivery addresses, apply a coupon, choose a delivery slot and place an order. Customers earn five loyalty points for every one hundred rupees spent; each point is worth one rupee at the time of redemption. An in-built AI assistant answers questions framed in ordinary language and returns matching

products along with a short explanation.

Administrators work in a separate panel protected by role-based access. From there they manage products and categories, move orders through the life cycle from placed to delivered, watch stock levels and receive low-stock warnings, study revenue and best-selling items, view new and repeat customers, configure the loyalty rate and inspect the loyalty ledger. A dedicated prediction screen estimates the next day revenue and order count and recommends how many units of each item should be baked. The POS screen handles walk-in sales at the counter, registers a new customer on the spot if necessary, deducts stock and updates loyalty points in a single transaction.

1.3 ORGANISATION OF THE REPORT

The remainder of this report is organised into seven chapters.

- **Chapter 2** presents the system analysis. It states the problems found in the existing manual method of working, lists the objectives the project sets out to achieve and defines precisely what lies inside and outside the scope of the work.
- **Chapter 3** records the hardware and software requirements, studies each technology selected for the implementation, and enumerates the functional and non-functional requirements of the system.
- **Chapter 4** describes the design of the system. It presents the layered architecture, the level 0 and level 1 data flow diagrams, the use case diagram, a sequence diagram for placing an order, an activity diagram for counter billing and the complete database design including the entity relationship diagram and collection schemas.
- **Chapter 5** explains the implementation. It documents the development environment, the folder structure, the description of every module, the front end and back end code organisation, the REST API, the demand prediction algorithm, the loyalty engine and the output screens of the application.
- **Chapter 6** deals with testing. It records the objectives of testing, the strategy followed at each level, the test cases executed and the results observed.
- **Chapter 7** discusses the results obtained and evaluates the system against the objectives stated in Chapter 2.
- **Chapter 8** concludes the report and lists the enhancements that can be taken up in the future.

CHAPTER 2
SYSTEM ANALYSIS

2.1 INTRODUCTION

System analysis is the stage at which the requirements of a proposed system are studied in detail so that the design that follows rests on a clear and complete understanding of the problem. This chapter converts the informal description given in Chapter 1 into a precise problem statement, a list of measurable objectives and a definition of the boundary of the project. The analysis was carried out by observing the way a neighbourhood bakery records its orders, issues bills and manages its stock, and by studying the features offered by existing commercial bakery websites.

2.2 PROBLEM STATEMENT

Small and medium sized bakeries continue to record their business on paper. The following difficulties were identified and form the problem statement of this project.

1. **No online presence.** The bakery has no catalogue on the Internet, so a customer cannot see which sweets, cakes or savouries are available, what they cost or what pack sizes they come in without visiting the shop or telephoning it. Business is therefore limited to walk-in trade and to customers who already know what they want.
2. **Manual, error-prone order recording.** Orders are written by hand into a register. Prices and pack sizes are recalled from memory, arithmetic is done mentally or on a calculator, and totals are frequently wrong. There is no record of the order once the register page is filled.
3. **No visibility of stock.** Stock is counted by walking to the shelf. During a busy evening the counter assistant may promise an item that has just been sold, and the shop discovers the shortage only when the customer arrives. In the other direction, slow-moving items are noticed only after they have passed their freshness window and must be discarded.
4. **No estimate of tomorrow's demand.** The quantity to be baked each morning is decided by guesswork. On a festival day this leads to stock-outs and lost sales; on an ordinary weekday it leads to unsold perishable stock that has to be written off. The bakery has no way of using its own past sales to forecast the next day.
5. **No customer records and no loyalty programme.** Because transactions are not captured against a customer identity, the shop cannot tell a first-time buyer from a regular one. Valued customers receive the same treatment as everybody else, and there is no mechanism to reward repeat purchases or to inform a customer about an offer.

6. **Slow counter billing.** Every sale at the counter is written and totalled by hand. When several customers are waiting, the queue grows and mistakes become more likely. Walk-in customers are not asked for their details because doing so would slow the queue further.
7. **No confirmation or delivery proof.** For an order sent out for delivery there is no acknowledgement that reaches the customer and no evidence that the parcel reached the right person. Disputes are settled by recollection rather than by record.
8. **Online and counter sales are never consolidated.** Even when a bakery does accept orders through a messaging application, those orders live in a chat thread that has nothing to do with the counter register. The owner therefore has no single figure for the day's revenue, no single list of best-selling items and no single view of the customer base.

In summary, the problem is the absence of a single integrated system that can sell both over the Internet and across the counter, keep stock accurate as each sale happens, remember every customer and every transaction, forecast the next day's requirement from past sales, and give the owner one dashboard from which the whole business can be understood. **Creamy Bites** is proposed as the solution to this problem.

2.3 OBJECTIVES OF THE PROJECT

The objectives listed below were framed so that each one answers a specific difficulty recorded in the problem statement.

1. To design and develop a responsive online storefront that presents the bakery catalogue by category with images, descriptions, pack sizes, prices, discounts and ratings.
2. To provide fast product discovery through keyword search, category filters and sorting by price, rating and novelty, and through a conversational AI assistant that answers questions written in ordinary language.
3. To implement secure user registration and login using JSON Web Tokens and bcrypt password hashing, together with role-based access that separates an ordinary customer from an administrator.
4. To build a persistent shopping cart and a wishlist that survive page reloads and are restored whenever the user signs in again.
5. To support a complete checkout flow with a saved address book, coupon application, choice of delivery slot, a minimum order value rule and a serviceable-city rule.
6. To generate a four-digit delivery OTP for every order and to hand the customer a ready-made WhatsApp receipt so that delivery can be verified and the customer has a written acknowledgement.

7. To give the administrator a dashboard that manages products and categories, moves orders through their life cycle, maintains inventory with low-stock warnings and maintains the list of customers.
8. To develop a Point-of-Sale module for counter billing that can look up an existing customer by name, phone or e-mail, register a walk-in customer in one step, deduct stock and record the sale immediately.
9. To design a loyalty programme that credits five points for every one hundred rupees spent, allows redemption at one rupee per point and maintains a full ledger of every earning and redemption.
10. To identify repeat customers automatically and to reward them with graded discounts of five, ten and fifteen percent based on how many orders they have placed.
11. To forecast the next day's revenue and order count using Exponentially Weighted Moving Average smoothing over twenty days of history, scaled by day-of-week and Indian festival multipliers, and to recommend baking quantities with a safety buffer.
12. To provide analytical reports on total revenue, average order value, best-selling products, category performance, daily sales and the split between new and repeat customers.
13. To keep the system maintainable and extensible by separating the user interface, the business logic and the data access layers, and by exposing all business functions through a documented REST API.

2.4 SCOPE OF THE PROJECT

2.4.1 In Scope

- A customer-facing web application covering registration, login, catalogue browsing, search, product details with reviews, cart, wishlist, address book, coupons, checkout, order history and profile management.
- An administrative web application covering the dashboard, product and category management, order management, inventory control, customer management, new and repeat customer reports, loyalty configuration and ledger, sales analytics, demand prediction and counter billing.
- Seven product categories, namely Sweets, Savouries, Cakes, Snacks, Cookies, Breads and Beverages, each holding five representative products with multiple pack sizes and prices.
- A REST API covering authentication, products, categories, cart, orders, wishlist, addresses, coupons, reviews, administration and point-of-sale billing.

- Persistence in MongoDB Atlas through Mongoose schemas, with a JSON file fallback so that the application continues to run if the cluster cannot be reached.
- A demand prediction engine, a loyalty engine, a repeat-customer engine and an AI shopping assistant.
- Delivery restricted to Chennai, a minimum online order value of four hundred rupees, and payment methods recorded as cash on delivery, UPI, card or cash at the counter.

2.4.2 Out of Scope

- Integration with a live payment gateway. The application records the chosen payment method but does not capture card or UPI credentials, and no money is actually transferred.
- Native Android or iOS applications. The user interface is a responsive web application that adapts to phone, tablet and desktop screens.
- Delivery outside the Chennai city limits, and real-time tracking of the delivery vehicle on a map.
- Direct integration with an SMS or e-mail service provider. The order receipt is handed to the customer as a pre-filled WhatsApp deep link that is opened by the customer's own application.
- Multi-store, multi-warehouse and franchise management, and accounting or statutory tax filing features.

2.4.3 Limitations

- The prediction module needs at least a few weeks of recorded orders before its output becomes dependable; with fewer than sixteen recorded orders the system reports a baseline confidence of seventy-five percent and falls back on catalogue averages.
- Product images are referenced from external content delivery URLs rather than being stored on the application server, so an Internet connection is needed for the catalogue to render completely.
- The application currently supports a single language and a single currency.

CHAPTER 3

SYSTEM REQUIREMENTS AND TECHNOLOGY STUDY

3.1 HARDWARE REQUIREMENTS

The minimum hardware needed to develop and to run the application is given in Table 3.1.

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 or equivalent (1.8 GHz)	Intel Core i5 or equivalent (2.4 GHz and above)
Main memory	4 GB RAM	8 GB RAM or more
Secondary storage	20 GB free disk space	50 GB free disk space (SSD preferred)
Display	1366 x 768 pixels	1920 x 1080 pixels
Input devices	Keyboard and pointing device	Keyboard, mouse and touch screen
Network	Broadband Internet connection, 2 Mbps	Broadband Internet connection, 10 Mbps or faster
Optional hardware	-	Bar-code scanner and thermal bill printer for the POS counter

Table 3.1 Hardware requirements

3.2 SOFTWARE REQUIREMENTS

The software used to build, run and test the application is listed in Table 3.2.

Layer	Software / Library	Version	Purpose
Operating system	Windows 10 / Windows 11 or any Linux distribution	64-bit	Development and deployment host
Runtime	Node.js	18.x LTS	Executes the back-end server
Package manager	npm	9.x	Installs and manages dependencies
Front-end library	React	18.3	Builds the user interface
Front-end build tool	Vite	5.1	Development server and production bundler
Routing	react-router-dom	6.22	Client-side page navigation
HTTP client	Axios	1.6	Calls the REST API from the browser
Back-end framework	Express	4.18	Defines routes and middleware
Database	MongoDB Atlas	6.x / 7.x	Stores all application data
ODM	Mongoose	8.x	Defines schemas and queries the database

Layer	Software / Library	Version	Purpose
Authentication	jsonwebtoken	9.0	Issues and verifies JSON Web Tokens
Password security	bcryptjs	2.4	Hashes and compares passwords
File upload	multer	1.4	Handles product image uploads
Validation	express-validator	7.0	Validates request payloads
Logging	morgan	1.10	Logs HTTP requests in development
Cross-origin support	cors	2.8	Permits requests from the front-end origin
Development aid	nodemon	3.0	Restarts the server on file changes
Browser	Google Chrome / Mozilla Firefox / Microsoft Edge	Latest	Runs and tests the application
Version control	Git and GitHub	Latest	Source code management

Table 3.2 Software requirements

3.3 TECHNOLOGY STUDY

3.3.1 React 18

React is a JavaScript library for building user interfaces from small, reusable components. Each component owns its state and renders itself again whenever that state changes, so the interface always reflects the current data without the developer writing any code to update the page by hand. React 18 was chosen because its component model keeps the storefront, the administration panel and the billing screen separate while sharing the same card, button and layout components.

3.3.2 Vite

Vite is the build tool and development server used for the front end. It serves source files natively during development, so the server starts almost instantly and a change to a file appears in the browser in a few milliseconds. Vite also proxies every request beginning with /api to the Express server, which removes the need for any cross-origin configuration while developing.

3.3.3 Node.js and Express

Node.js runs JavaScript outside the browser on the V8 engine, which lets the same language be used on the client and on the server. Its event-driven, non-blocking input-output model suits an application that spends most of its time waiting for database queries. Express is a thin framework that sits on top of Node.js and supplies routing, middleware chaining and error handling, which keeps the server code organised into controllers.

3.3.4 MongoDB and Mongoose

MongoDB stores data as flexible JSON-like documents instead of fixed rows, which suits a product catalogue in which items carry different pack sizes and optional attributes. Mongoose adds schemas, validation, type casting and population of references on top of the raw driver, so the application works with predictable objects rather than untyped dictionaries.

3.3.5 JSON Web Token

A JSON Web Token is a compact, signed string that carries the identity and the role of the logged-in user. The server issues a token at login and the browser attaches it to every subsequent request. Because the signature can be verified without a database lookup, the server stays stateless and scales easily. The token is set to expire after seven days.

3.3.6 bcryptjs

Passwords are never stored in plain text. bcryptjs hashes each password with a random salt and a deliberately slow key-derivation function, so two users who choose the same password get different stored values and an attacker who steals the database cannot recover the original passwords quickly.

3.3.7 Axios

Axios is the HTTP client used by the front end. A single configured instance holds the base address and an interceptor that attaches the stored token to every request, and a response interceptor that unwraps the payload and rejects with a readable error message. This keeps every screen free of repetitive networking code.

3.3.8 Multer and Morgan

Multer parses multipart form data so that product images can be uploaded to the server and served again from the /uploads path. Morgan prints one line per HTTP request during development, which makes it easy to see which endpoint was called and how long it took.

3.4 FUNCTIONAL REQUIREMENTS

The functional requirements describe what the system must do. They are grouped by module.

- **Authentication module.** Register a new customer with name, e-mail, phone and password; reject an e-mail that is already registered; log a user in and return a token; return and update the signed-in profile; block every protected route when no valid token is supplied.
- **Catalogue module.** List categories with icons and images; list products with pagination; return a chosen product by identifier or by slug; list featured, bestselling, new and discounted products; search by keyword across name, description and category.

- **Cart module.** Add an item with a chosen pack size; change the quantity; remove a line; clear the whole cart; return the current cart with computed line totals and a cart total.
- **Wishlist module.** Add or remove a product with a single toggle; list the saved products of the signed-in user.
- **Address module.** Create, list and delete delivery addresses; mark one address as the default.
- **Coupon module.** Accept a coupon code and the current cart total and return the discount earned, supporting both percentage and flat discounts.
- **Order module.** Place an order from the current cart; group duplicate lines by product and pack size; enforce the minimum order value and the serviceable city; apply a manual coupon or an automatic loyalty discount; generate a delivery OTP; list the orders of a user and return one order in detail; build the WhatsApp receipt text and link.
- **Review module.** List the reviews of a product and allow a signed-in user to add a rating and a comment.
- **Administration module.** Create, update and delete products; list all orders and change the status of an order; list customers; list inventory and update stock; read and update the loyalty configuration; read the loyalty ledger; return analytics; return the demand prediction; list categories.
- **Point-of-sale module.** Search for a customer by name, phone or e-mail and return the loyalty balance; register a walk-in customer from a name and a phone number alone; create a counter sale that validates and deducts stock, redeems points, computes earned points, writes an order of type pos and updates the loyalty ledger.
- **Prediction module.** For a given date return the predicted revenue, the predicted number of orders, a confidence score, the top five products and the recommended baking quantities, together with the reasoning behind the numbers.
- **Assistant module.** Accept a natural-language question and return a short answer and up to six matching products, with dedicated answers for questions about loyalty, delivery and dietary preferences.

3.5 NON-FUNCTIONAL REQUIREMENTS

- **Security.** Passwords are hashed with bcrypt and never returned by any endpoint. Protected routes require a valid bearer token, administrative routes additionally require the admin role, and input is validated before it reaches the database.
- **Performance.** The catalogue is served from a single indexed query, the browser keeps cart and wishlist state in memory between navigations, and the front end is bundled and minified for production so that the first screen loads quickly.

- **Usability.** The interface is responsive from a phone to a desktop, uses familiar e-commerce patterns, shows a toast message after every significant action and disables a control while its request is in flight.
- **Reliability.** If the MongoDB cluster cannot be reached the server falls back to a local MongoDB instance or to a JSON file store, so the shop can continue to take orders.
- **Maintainability.** The code is separated into models, controllers, routes and middleware on the server and into pages, components, contexts and services on the client, so a change to one feature does not disturb the others.
- **Scalability.** The server is stateless and the database is hosted, so additional instances can be placed behind a load balancer without any change to the application.
- **Portability.** Both tiers run on Windows, Linux and macOS and depend only on Node.js and a browser.

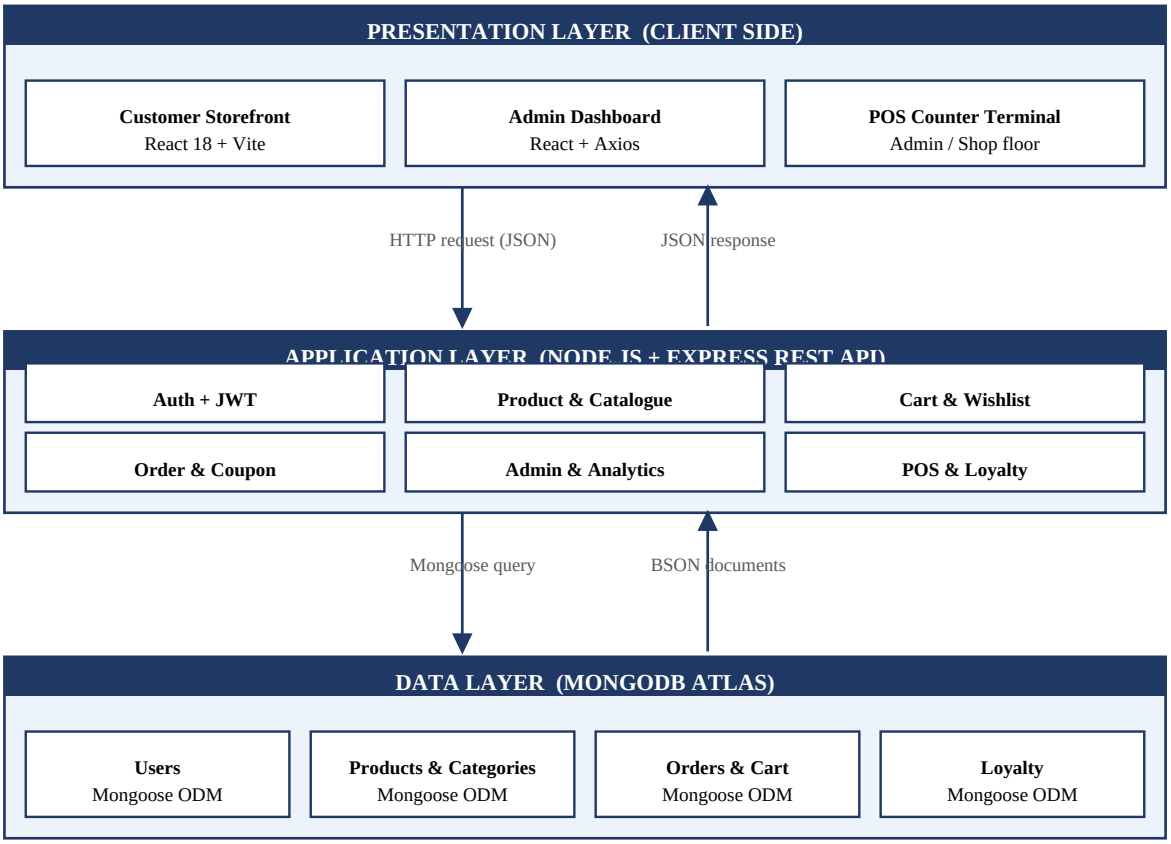
CHAPTER 4
SYSTEM DESIGN

4.1 INTRODUCTION

This chapter presents the design of Creamy Bites. The design is described at four levels of detail: the overall architecture, the flow of data through the system, the behaviour of the system as seen by its actors, and the structure of the data that the system stores. Each level is supported by a diagram drawn with standard notation.

4.2 SYSTEM ARCHITECTURE

The application follows a three-tier architecture, shown in Figure 4.1. The presentation tier is the React single-page application that runs in the browser. It never talks to the database directly; every request goes through the application tier. The application tier is the Express server, which receives a request, passes it through the authentication middleware when the route is protected, hands it to the relevant controller, and returns a JSON document. The data tier is the MongoDB Atlas cluster, reached through Mongoose models. Because the three tiers are independent, the interface can be redesigned without touching the business rules, and the database can be moved without touching the interface.



Cross-cutting services: JWT authentication, CORS policy, Morgan request logging, Multer image upload, bcrypt password hashing
Every protected request carries an "Authorization: Bearer <JWT>" header; the server stays stateless.

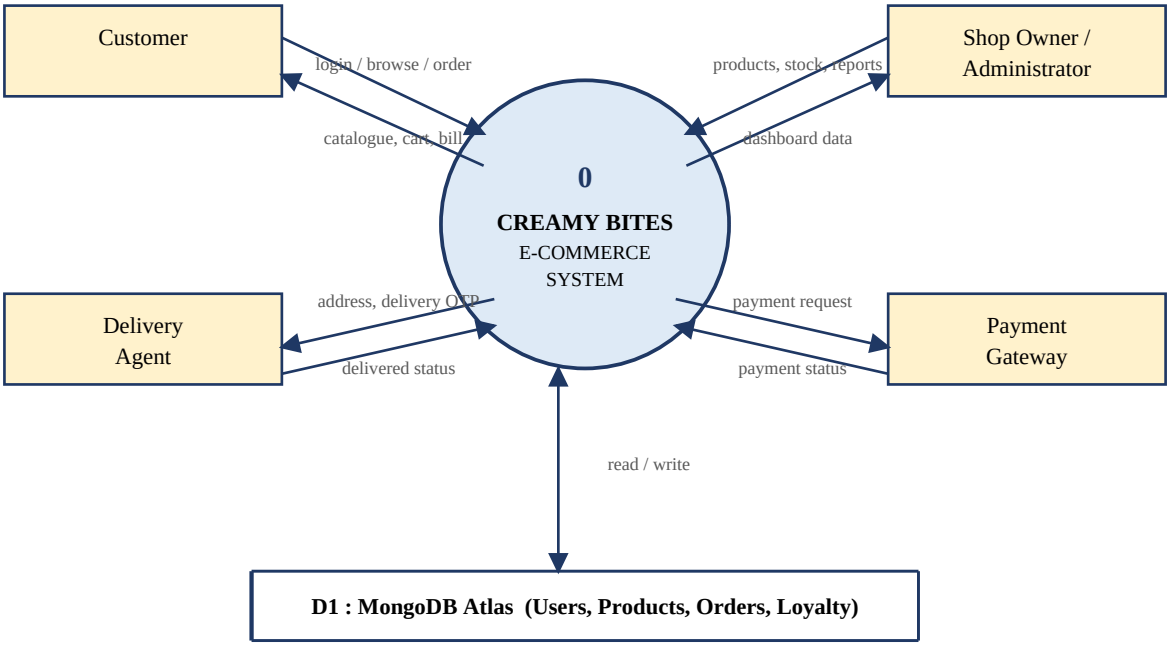
Fig. 4.1 Three-tier architecture of the Creamy Bites platform

Two decisions are worth recording. First, all communication between the browser and the server uses JSON over HTTP, so the same API can later serve a mobile application. Second, the server holds no session state; the token presented with each request carries the identity and the role of the caller, which is what allows the application tier to be scaled horizontally.

4.3 DATA FLOW DIAGRAMS

4.3.1 Level 0 Data Flow Diagram

Figure 4.2 is the context diagram. The whole system is drawn as a single process, process 0, and the four external entities that exchange data with it are shown around it. The customer supplies login credentials, browse requests and orders, and receives the catalogue, the cart contents and the bill. The shop owner supplies product, stock and report requests and receives dashboard data. The delivery agent receives the address and the delivery OTP and returns the delivered status. The payment gateway receives a payment request and returns the payment status. All four flows are persisted in the single data store D1.

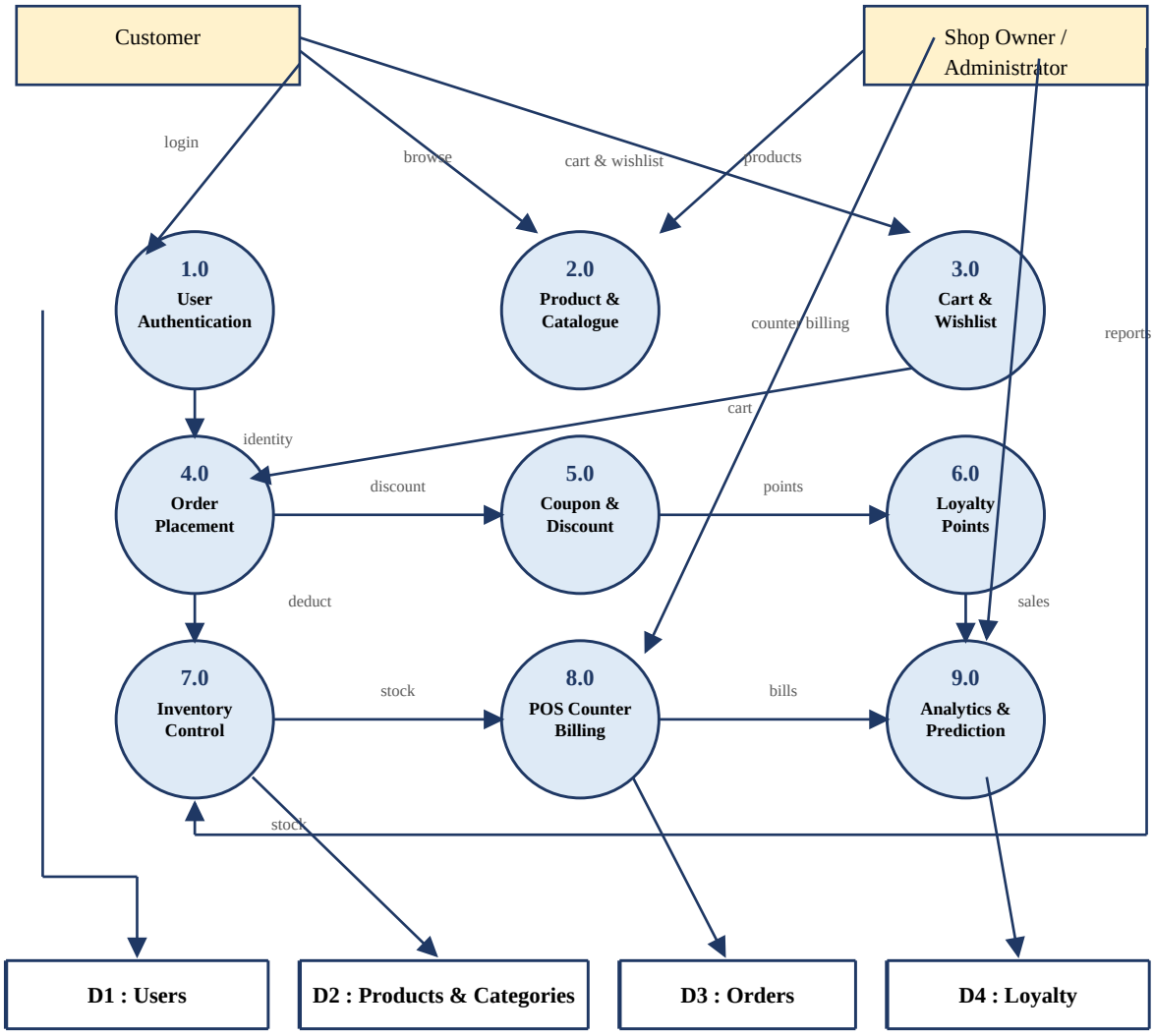


Level 0 (Context) Data Flow Diagram

Fig. 4.2 Level 0 data flow diagram (context diagram)

4.3.2 Level 1 Data Flow Diagram

Figure 4.3 decomposes process 0 into nine processes and six data stores. Process 1.0 authenticates users and writes to D1. Process 2.0 manages the catalogue and reads and writes D2 and D3. Process 3.0 maintains the cart and the wishlist in D4. Process 4.0 places orders and writes them to D5. Process 5.0 evaluates coupons and passes the discount to process 4.0. Process 6.0 maintains the loyalty ledger in D6. Process 7.0 controls stock in D3. Process 8.0 handles counter billing and writes to D5 and D6. Process 9.0 reads the orders in D5 to produce analytics and the demand prediction.

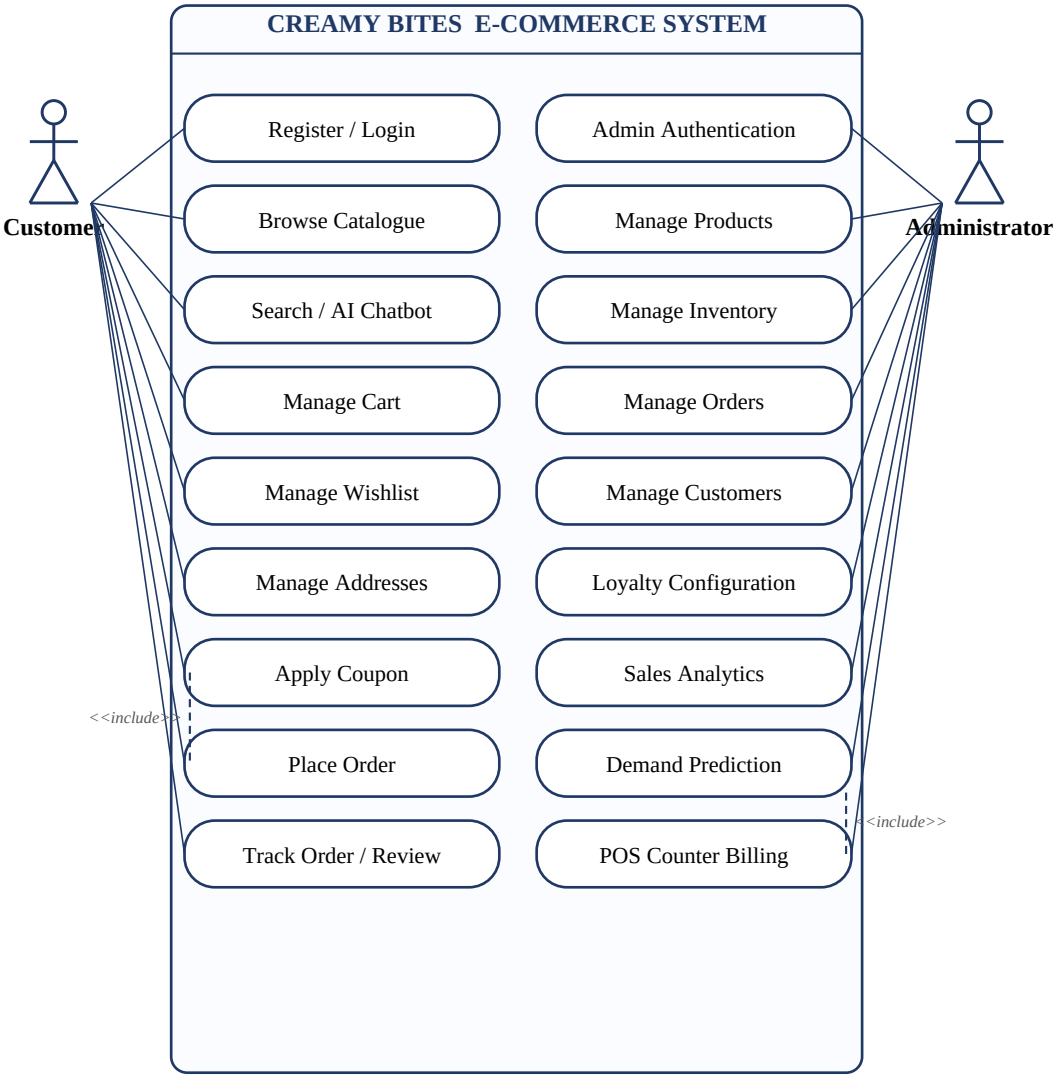


Level 1 Data Flow Diagram - detailed processes and data stores

Fig. 4.3 Level 1 data flow diagram

4.4 USE CASE DIAGRAM

Figure 4.4 shows the two actors of the system and the use cases available to each of them. The customer registers and logs in, browses the catalogue, searches or asks the assistant, manages a cart, a wishlist and an address book, applies a coupon, places and tracks an order and writes a review. The administrator authenticates separately, manages products, inventory, orders and customers, configures the loyalty programme, studies analytics, requests a demand prediction and bills a walk-in customer at the counter. Two include relationships are drawn: applying a coupon is included in placing an order, and POS billing includes the authentication of the administrator.

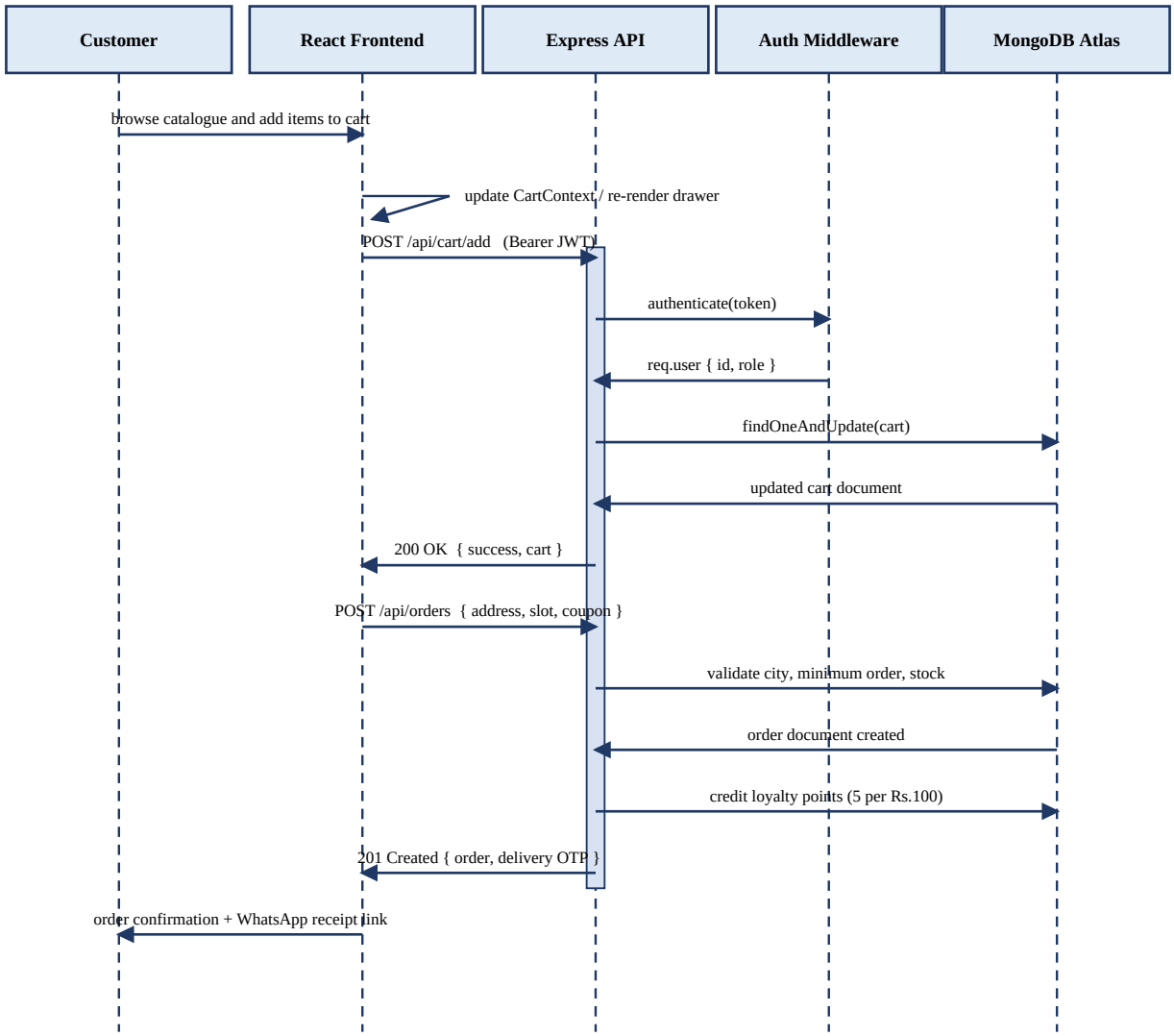


Actors, use cases and include-relationships of the proposed system

Fig. 4.4 Use case diagram of the proposed system

4.5 SEQUENCE DIAGRAM

Figure 4.5 traces the messages exchanged when a customer places an online order, which is the most important interaction in the system. The customer acts on the React front end; the front end calls the Express API with the stored token; the API passes the token to the authentication middleware and receives the decoded user back; the API then reads and writes MongoDB and returns a response. Once the order is created the API credits loyalty points in a second write and returns the order together with the delivery OTP, and the front end shows a confirmation with the WhatsApp receipt link.



Sequence of interactions while placing a secured online order

Fig. 4.5 Sequence diagram for placing a secured online order

4.6 ACTIVITY DIAGRAM

Figure 4.6 models the flow of a counter sale through the Point-of-Sale module. The assistant first looks the customer up; if the customer is not found, a walk-in record is created from a name and a phone number alone, and if the customer is found the loyalty balance is loaded. Products are then added to the bill. Each line is checked against available stock and rejected with a clear message when stock is short. The assistant may then redeem loyalty points at one rupee per point, after which the subtotal, the discount and the payable amount are computed. When payment is received, stock is deducted, the order is written with the type pos, points are credited and the bill is printed; if payment is not received the bill is held.

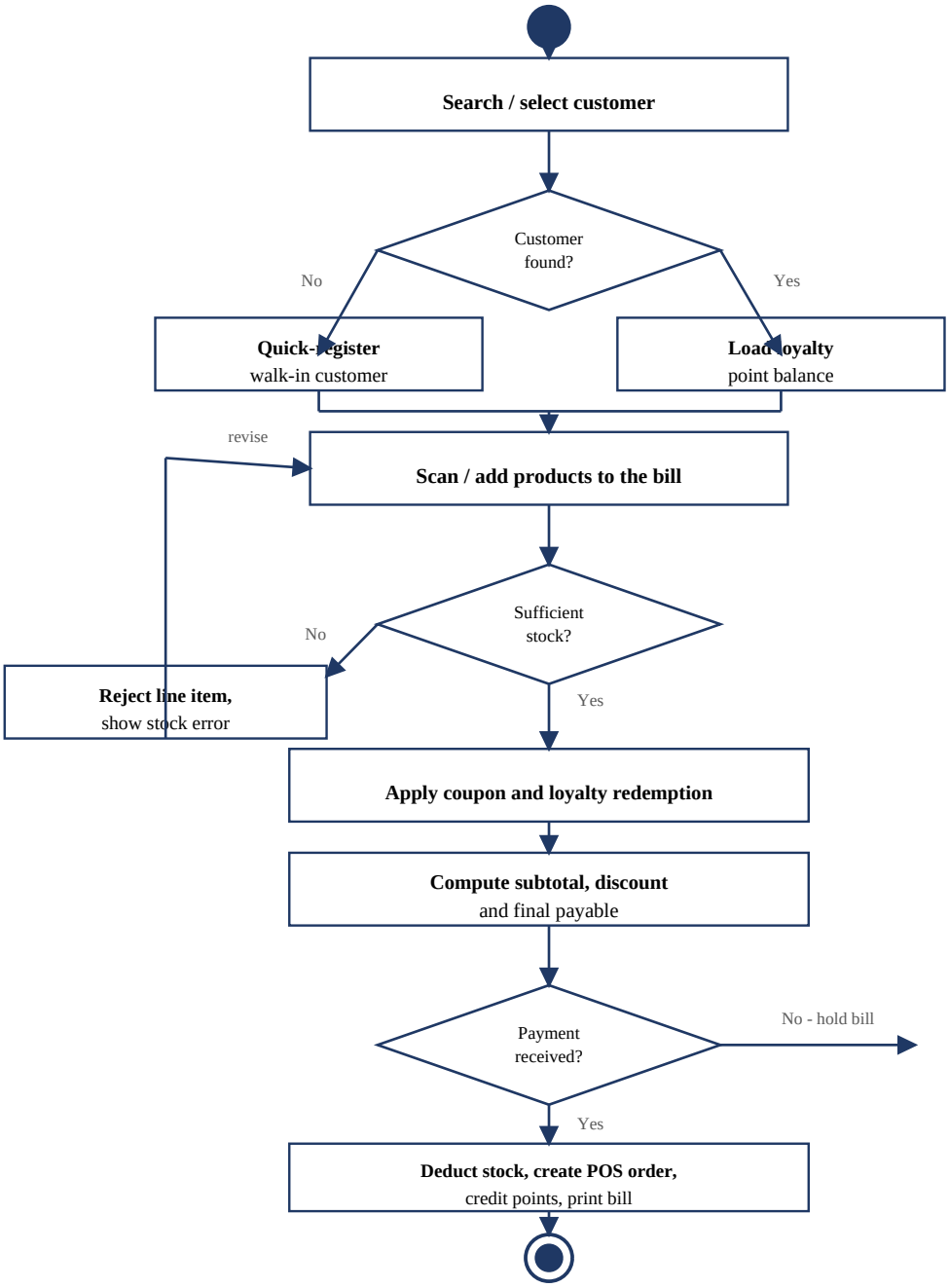


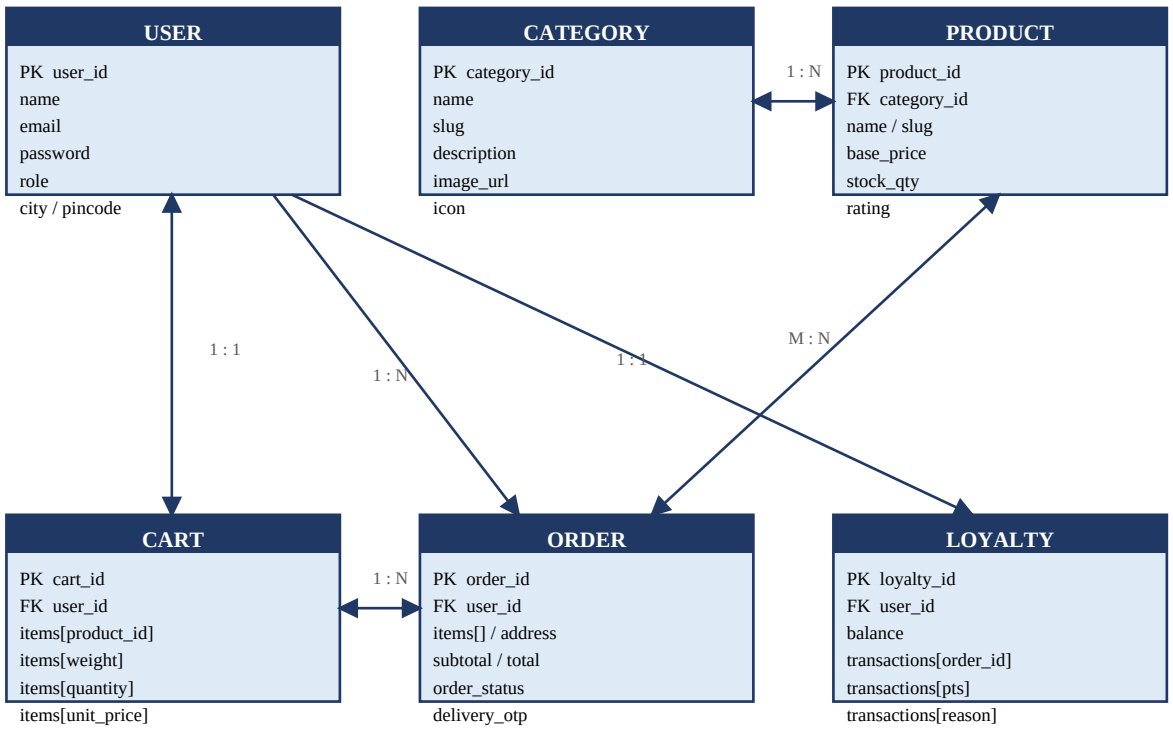
Fig. 4.6 Activity diagram for the Point-of-Sale billing flow

4.7 DATABASE DESIGN

4.7.1 Entity Relationship Diagram

Figure 4.7 shows the entities of the system and the relationships between them. A user owns exactly one cart and exactly one loyalty account, may save many addresses and may place many orders. A category holds many products, and an order contains many products while a product appears in many orders, which is a many-to-many relationship resolved by embedding the line items inside the order document. Two further collections, addresses and

reviews, are maintained for delivery addresses and product ratings.



Cardinality: one user owns one cart and one loyalty account, and places many orders; each category holds many products and each order contains many products (M : N).

PK = primary key (_id) *FK = foreign key reference* *Additional collections: addresses, reviews, coupons*

Fig. 4.7 Entity relationship diagram of the Creamy Bites database

4.7.2 Summary of Collections

Table 4.1 lists every collection with the purpose it serves.

Collection	Purpose	Important Fields
users	Stores customers and administrators	name, email, password hash, phone, role, city
categories	The seven product categories	name, slug, description, image_url, icon
products	The bakery catalogue	category_id, name, slug, base_price, stock_qty, weights[]
carts	One persistent cart per user	user_id, items[]
orders	Online orders and counter sales	user_id, items[], address, total, order_status, order_type
addresses	Saved delivery addresses	user_id, name, phone, city, pincode
loyalties	Loyalty balance and transaction ledger	user_id, balance, transactions[]
reviews	Product ratings and comments	product_id, user_id, rating, comment

Table 4.1 Summary of MongoDB collections

4.7.3 Schema of the User Collection

Table 4.2 gives the schema of the users collection.

Field	Type	Constraints	Description
_id	ObjectId	Primary key	Unique identifier generated by MongoDB
name	String	Required	Full name of the user
email	String	Required, unique, lower-case	Used as the login identifier
password	String	Required	bcrypt hash, never returned by the API
phone	String	Default empty	Contact number, used for POS lookup and WhatsApp
role	String	Enum: customer, admin	Determines access to administrative routes
address_line1 / 2	String	Optional	Default postal address
city / state	String	Default Chennai / Tamil Nadu	Serviceable location
pincode	String	Optional	Postal code
createdAt / updatedAt	Date	Automatic	Timestamps maintained by Mongoose

Table 4.2 Schema of the users collection

4.7.4 Schema of the Product Collection

Table 4.3 gives the schema of the products collection.

Field	Type	Description
_id	ObjectId	Primary key
category_id	ObjectId	Reference to the categories collection
name	String	Product name, for example Chocolate Truffle Cake
slug	String	Unique URL-friendly name used in the product page route
description	String	Short description shown on the product card and page
image_url	String	Address of the product photograph
base_price	Number	Price of the smallest pack size
discount_percent	Number	Discount applied to the displayed price
is_bestseller / is_new / is_offer	Number (0 or 1)	Badges shown on the product card
rating / rating_count	Number	Average customer rating and number of ratings
stock_qty	Number	Units currently available; default 30
low_stock_threshold	Number	Level at which the item is flagged; default 10
weights	Array of {weight, price}	Available pack sizes and their prices
is_active	Number (0 or 1)	Allows a product to be hidden without deletion

Table 4.3 Schema of the products collection

4.7.5 Schema of the Order Collection

Table 4.4 gives the schema of the orders collection.

Field	Type	Description
_id	ObjectId	Primary key; the last six characters form the order number
user_id	ObjectId	Reference to the users collection
items	Array	Embedded line items with product, pack size, quantity and price
address	Object	Snapshot of the delivery address at the time of ordering
coupon_code	String	Coupon or loyalty code applied to the order
subtotal / discount_amount	Number	Amount before and after the discount
delivery_charge / total	Number	Delivery charge and the final payable amount
payment_method	String	cod, upi, card or cash
delivery_slot / notes	String	Chosen delivery window and customer instructions
order_status	String	Enum: placed, confirmed, preparing, packed, out_for_delivery, delivered, cancelled
order_type	String	Enum: online or pos
payment_status	String	Enum: pending or paid
points_earned / points_redeemed	Number	Loyalty points credited and used
delivery_otp	String	Four-digit code used to confirm delivery

Table 4.4 Schema of the orders collection

CHAPTER 5

IMPLEMENTATION

5.1 INTRODUCTION

This chapter describes how the design of Chapter 4 was turned into working software. It records the development environment, the layout of the source tree, the purpose of every module, the organisation of the front-end and the back-end code, the REST interface, the two algorithms that give the system its distinctive behaviour, and finally the screens that the system produces.

5.2 DEVELOPMENT ENVIRONMENT AND PROJECT STRUCTURE

The project is developed as two independent packages that can be installed and started separately. The back end listens on port 5000 and the Vite development server listens on port 5173 and forwards every request whose path begins with /api to the back end. The source tree is given below.

```
Minicapstone-Projecct/
|-- backend/
|   |-- data/db.json                  JSON fallback store
|   |-- scripts/                      one-time seeding and clean-up scripts
|   `-- src/
|       |-- server.js                 Express application and route mounting
|       |-- config/db.js              MongoDB connection with local fallback
|       |-- middleware/auth.js        authenticate, optionalAuth, requireAdm
in
|       |-- models/                   User, Product, Category, Cart, Order,
|       |                             Address, Loyalty, seed, db fallback
|       |-- controllers/              auth, product, category, cart, order,
|       |                             wishlist, address, coupon, review,
|       |                             admin, pos
|       `-- routes/                   one router per resource
`-- frontend/
    |-- index.html
    |-- vite.config.js                dev server and /api proxy
    `-- src/
        |-- main.jsx  App.jsx         application entry and routing
        |-- services/api.js           Axios instance and API wrappers
        |-- context/                  Auth, Cart, Wishlist, Toast providers
        |-- components/               Navbar, Footer, HeroBanner,
        |                             CategoryGrid, ProductCard,
        |                             CartDrawer, AIChatbot
        `-- pages/
            |-- Home/ ProductListing/ ProductDetails/
            |-- Cart/ Checkout/ Login/ Register/
            |-- MyOrders/ Wishlist/ Profile/
            `-- Admin/                Dashboard, Products, Orders,
                                      Customers, Inventory, Loyalty,
                                      Analytics, Prediction, POS
```

5.3 MODULE DESCRIPTION

5.3.1 Authentication and Authorisation Module

Registration collects a name, an e-mail address, a phone number and a password, rejects an e-mail that is already registered, hashes the password with bcrypt at cost factor ten and creates the user with the role customer. If an address was typed during registration it is saved as the default address in the same step. Login compares the supplied password with the stored hash and, on success, returns a JSON Web Token that carries the identifier and the role of the user and expires after seven days. Two middleware functions protect the routes: authenticate verifies the bearer token and loads the user, and requireAdmin refuses the request unless the role is admin.

5.3.2 Category and Product Catalogue Module

Categories are stored with a name, a slug, a description, an icon and a display order, and are returned to the home page as a grid. Products belong to a category and carry a slug, a

description, an image, a base price, a discount percentage, a set of badge flags, a rating, a stock quantity and an array of pack sizes with individual prices. The controller offers listing by category, lookup by identifier or slug, featured and bestselling lists, keyword search and natural-language queries.

5.3.3 Search and AI Assistant Module

Keyword search matches the question against the product name, the description and the category name and returns the closest matches. The assistant built on top of the same search first looks for an informational intent: a question about loyalty, points or rewards is answered with a description of the programme, a question about delivery or timing is answered with the delivery policy, and a question about eggless, vegetarian or sugar-free items is answered with a suitable selection from the catalogue. Any other question is treated as a product query and answered with the matching items and a short recommendation.

5.3.4 Cart Module

Each user owns one cart document holding an array of line items. A line records the product, the chosen pack size, the name and image at the time of adding, the unit price, the quantity and the computed line total. Adding the same product and pack size again increases the quantity instead of creating a duplicate line. The cart survives a reload because it is fetched from the server whenever the application starts and the user is signed in.

5.3.5 Wishlist Module

The wishlist stores one record per user and product. A single endpoint toggles the entry, so the heart control on a product card needs to make only one call and the interface can update optimistically.

5.3.6 Address Book Module

A user may save any number of addresses with a name, a phone number, two address lines, a city, a state, a pin code and a label such as Home or Work. One address may be marked as the default and is pre-selected at checkout.

5.3.7 Coupon and Discount Module

Three coupon codes are supported: two percentage discounts and one flat discount. The endpoint receives the code and the current cart total and returns the amount of the discount, which the interface then shows as a separate line in the bill.

5.3.8 Order Placement and Tracking Module

Placing an order reads the cart, groups duplicate lines by product and pack size, computes the subtotal and then applies three business rules in order: the subtotal must reach the minimum online order value, the delivery city must be serviceable, and a discount is applied from either the coupon typed by the customer or the automatic repeat-customer offer. The order is written

with a four-digit delivery OTP, and the response includes a pre-filled WhatsApp link carrying the order number, the amount, the OTP and the points that will be earned. The order history screen lists the orders of the user and shows the current status of each one.

5.3.9 Loyalty Points Module

Every completed order earns five points for each full one hundred rupees of the final payable amount. Redemption converts points into a discount at one rupee per point, limited to the smaller of the available balance and the rounded subtotal. Each earning and each redemption is appended to the transactions array of the loyalty document with the order reference, the number of points, the reason and the date, so the complete ledger can be displayed to the administrator.

5.3.10 Repeat Customer Module

The administrator dashboard counts how many non-cancelled orders each customer has placed and classifies them into four groups: new customers with a single order, loyal customers with three to four orders, regular customers with five to nine orders and VIP customers with ten or more. Separate screens list the new customers and the repeat customers, and the repeat-customer screen ranks them by the number of orders and by the value they have brought.

5.3.11 Inventory Management Module

Stock is stored on the product document along with a low-stock threshold, which defaults to ten units. The inventory screen lists every product with its current stock, highlights the items at or below the threshold and allows the administrator to raise or correct the quantity. Stock is deducted inside the order placement flow and inside the counter billing flow, so the two channels can never sell the same unit twice.

5.3.12 Analytics and Demand Prediction Module

The analytics screen reports total revenue, order count and average order value, the revenue by day, the performance of each category and the best selling products. The prediction screen looks twenty days back, groups the historical orders into daily totals, smooths them with an Exponentially Weighted Moving Average and scales the result by a multiplier that depends on the day of the week and on whether the target date is a festival. It returns the predicted revenue, the predicted number of orders, a confidence score, the top five products and the recommended baking quantities.

5.3.13 Point-of-Sale Module

The counter screen lets the assistant search for a customer by name, phone number or e-mail and see the loyalty balance immediately. A walk-in customer who is not found can be registered from a name and a phone number alone, with a generated internal e-mail address.

The sale itself validates each line against available stock, deducts the stock, optionally redeems points, computes the earned points, writes an order with the type pos and the status delivered, and updates the loyalty ledger.

5.3.14 Administration Panel Module

The administration area is a separate layout with its own side navigation and is wrapped in a route guard that redirects a non-administrator to the store front. It hosts the dashboard, product management, order management, customer management, the new and repeat customer reports, inventory, loyalty configuration and ledger, analytics, prediction and counter billing.

5.4 FRONT-END IMPLEMENTATION

5.4.1 Component Organisation

The user interface is composed of components that are reused across screens. The navigation bar carries the logo, the search field, the wishlist and cart counters and the account menu. The hero banner advertises the current offer, the category grid links to each product category, and the product card displays the image, the name, the rating, the price with the struck-through original price when a discount applies, the pack-size selector and the badges for bestseller, new and offer. The cart drawer slides in from the right so a shopper can review and change the cart without leaving the page. The AI assistant is a floating panel that shows a short conversation and renders product suggestions as cards that can be added to the cart in one click.

5.4.2 State Management

Four React contexts hold the state that more than one screen needs. AuthContext keeps the signed-in user and the token, restores them from local storage when the application starts and signs the user out when the token is rejected. CartContext keeps the cart lines, exposes add, update, remove and clear operations and keeps the drawer open or closed. WishlistContext keeps the set of saved product identifiers. ToastContext holds a small queue of messages so that any screen can report success or failure in the same way. Because the state lives in a context, the cart badge in the navigation bar updates as soon as a product is added from anywhere in the application.

5.4.3 Routing

React Router defines two areas. The store front occupies the root and includes the home page, the listing page, the product page reached by slug, the cart, the checkout, the login and registration screens, the order history, the wishlist and the profile. The administration area occupies /admin and is wrapped in a guard component that checks the role before rendering any of its children. Deep links work because every significant entity is addressed by a stable

slug or identifier in the address.

5.5 BACK-END IMPLEMENTATION AND REST API

The server is organised as a set of routers, one per resource, each delegating to a controller. Authentication and administrative routers mount the appropriate middleware for the whole router, so a protected endpoint cannot be reached by accident. Every controller returns the same envelope, a success flag together with either the data or a message, which lets the front end handle a response without inspecting the status code. A single error handler at the end of the chain converts an exception into a well-formed JSON error. Table 5.1 lists the endpoints exposed by the application.

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Create a customer account
POST	/api/auth/login	Public	Return a JSON Web Token
GET	/api/auth/me	User	Return the signed-in profile
PUT	/api/auth/profile	User	Update the profile
GET	/api/categories	Public	List the categories
GET	/api/products	Public	List products, optionally by category
GET	/api/products/featured	Public	List featured products
GET	/api/products/search?q=	Public	Keyword search
POST	/api/products/chat-query	Public	Natural-language query
GET	/api/products/slug/:slug	Public	Product detail by slug
GET	/api/products/:id	Public	Product detail by identifier
GET	/api/cart	User	Return the current cart
POST	/api/cart/add	User	Add an item
PUT	/api/cart/item/:itemId	User	Change a quantity
DELETE	/api/cart/item/:itemId	User	Remove a line
DELETE	/api/cart/clear	User	Empty the cart
GET	/api/wishlist	User	List saved products
POST	/api/wishlist/toggle	User	Add or remove a product
GET	/api/addresses	User	List saved addresses
POST	/api/addresses	User	Add an address
DELETE	/api/addresses/:id	User	Delete an address
POST	/api/coupons/apply	User	Validate a coupon
GET	/api/orders/loyalty-offer	User	Return the automatic loyalty offer
POST	/api/orders	User	Place an order
GET	/api/orders	User	List the orders of the user

Method	Endpoint	Access	Purpose
GET	/api/orders/:id	User	Return one order
GET	/api/reviews/:productId	Public	List reviews of a product
POST	/api/reviews/:productId	User	Add a rating and a comment
GET	/api/admin/dashboard	Admin	Summary cards and repeat statistics
GET	/api/admin/products	Admin	List products
POST	/api/admin/products	Admin	Create a product
PUT	/api/admin/products/:id	Admin	Update a product
DELETE	/api/admin/products/:id	Admin	Delete a product
GET	/api/admin/orders	Admin	List every order
PUT	/api/admin/orders/:id/status	Admin	Change the order status
GET	/api/admin/customers	Admin	List customers
GET	/api/admin/inventory	Admin	List stock levels
PUT	/api/admin/inventory/:id	Admin	Correct the stock
GET	/api/admin/loyalty/config	Admin	Read the earning rate
PUT	/api/admin/loyalty/config	Admin	Update the earning rate
GET	/api/admin/loyalty/ledger	Admin	Read the loyalty ledger
GET	/api/admin/analytics	Admin	Revenue and best-seller reports
GET	/api/admin/prediction	Admin	Next-day demand prediction
GET	/api/pos/customers/search	Admin	Look up a counter customer
POST	/api/pos/customers/quick-register	Admin	Register a walk-in customer
POST	/api/pos/sale	Admin	Record a counter sale
GET	/api/health	Public	Service health check

Table 5.1 REST API endpoints of the application

5.6 DEMAND PREDICTION ALGORITHM

The prediction module answers a question the bakery asks every evening: how much should be baked for tomorrow. It uses Exponentially Weighted Moving Average smoothing, which is appropriate here because it reacts quickly to a change in demand while remaining simple enough to run on every request.

5.6.1 Steps of the Algorithm

```
INPUT   target date (defaults to tomorrow)

1.  Read the last 100 non-cancelled orders, newest first.
2.  For each of the last 20 days d  (d = 0 is today):
        dailyRevenue[d]  =  sum of order.total  for orders on day d
        dailyOrders[d]   =  count of orders on day d
3.  Smooth each series with EWMA, weighting factor alpha = 0.3:
        EWMA  =  alpha * value[today]  +  (1 - alpha) * EWMA
    (the loop runs backwards so recent days carry the most weight)
4.  Choose the demand multiplier for the target date:
        1.45  if the date is a listed festival (Diwali, Pongal, ...)
        1.30  if the date is Friday, Saturday or Sunday
        1.20  if the date is Thursday
        1.05  for any other weekday
5.  predictedRevenue  =  round( EWMA(revenue) * multiplier )
    predictedOrders   =  ceil ( EWMA(orders)  * multiplier )
6.  For the five best-selling items:
        recommendedQty = ceil( avgDailyQty * multiplier * 1.25 )
    (the factor 1.25 adds a 25 percent safety buffer)
7.  Confidence  =  88% (High)      when more than 15 orders on record
                   80% (Moderate)   when 6 to 15 orders on record
                   75% (Baseline)   otherwise, using catalogue averages

OUTPUT  revenue, orders, confidence, top products, baking quantities,
        and the reasoning behind the numbers
```

5.6.2 Demand Multipliers

Table 5.2 shows the multipliers applied to the smoothed baseline.

Condition	Multiplier	Increase	Reason
Festival such as Diwali, Pongal, Christmas	1.45	+45%	Sweet and cake demand peaks on festival days
Friday, Saturday or Sunday	1.30	+30%	Weekend celebrations and bulk orders
Thursday	1.20	+20%	Traditional sweets demand for the weekly observance
Any other weekday	1.05	+5%	Steady baseline demand

Table 5.2 Demand multipliers used by the prediction module

5.7 LOYALTY AND REPEAT-CUSTOMER ENGINE

Two rules drive the loyalty engine. A customer earns five points for every full one hundred rupees of the final payable amount, and a point is worth exactly one rupee when it is redeemed. Redemption is limited to the smaller of the available balance and the rounded subtotal, so the payable amount can never become negative. Because the same engine serves both online orders and counter sales, a customer earns and spends points whichever channel is used.

Independently of points, the order placement flow counts how many non-cancelled orders the customer has already placed and applies an automatic discount when the customer has not typed a coupon of their own. The tiers are given in Table 5.3.

Orders Completed	Tier	Automatic Discount	Coupon Code Applied
10 or more	VIP customer	15% of the subtotal	LOYAL15
5 to 9	Regular customer	10% of the subtotal	LOYAL10
3 to 4	Loyal customer	5% of the subtotal	LOYAL5
1 to 2	New customer	No automatic discount	None

Table 5.3 Repeat-customer discount tiers

5.8 OUTPUT SCREENS

The screens produced by the application are reproduced in this section. The customer-facing screens are shown first, followed by the administration and counter screens.

INSERT SCREENSHOT

Home page showing the hero banner and the category grid

(paste your screen capture here)

Fig. 5.1 Home Page of the Creamy Bites store front

INSERT SCREENSHOT

Product listing page with filters and sorting

(paste your screen capture here)

Fig. 5.2 Product Listing Page

INSERT SCREENSHOT

Product details page with pack sizes and reviews

(paste your screen capture here)

Fig. 5.3 Product Details Page

INSERT SCREENSHOT

Shopping cart page with quantity controls and totals

(paste your screen capture here)

Fig. 5.4 Shopping Cart Page

INSERT SCREENSHOT

Checkout page with address, delivery slot and payment method

(paste your screen capture here)

Fig. 5.5 Checkout Page

INSERT SCREENSHOT

Login page

(paste your screen capture here)

Fig. 5.6 Login Page

INSERT SCREENSHOT

Registration page

(paste your screen capture here)

Fig. 5.7 Registration Page

INSERT SCREENSHOT

Order confirmation with the delivery OTP and the WhatsApp receipt link

(paste your screen capture here)

Fig. 5.8 Order Confirmation Screen

INSERT SCREENSHOT

My Orders page showing the status of each order

(paste your screen capture here)

Fig. 5.9 My Orders Page

INSERT SCREENSHOT

Wishlist page

(paste your screen capture here)

Fig. 5.10 Wishlist Page

INSERT SCREENSHOT

User profile page

(paste your screen capture here)

Fig. 5.11 User Profile Page

INSERT SCREENSHOT

AI shopping assistant answering a natural-language query

(paste your screen capture here)

Fig. 5.12 AI Shopping Assistant

5.8.1 Administration and Counter Screens

INSERT SCREENSHOT

Administration dashboard with summary cards and repeat-customer statistics

(paste your screen capture here)

Fig. 5.13 Administration Dashboard

INSERT SCREENSHOT

Product management screen

(paste your screen capture here)

Fig. 5.14 Product Management Screen

INSERT SCREENSHOT

Order management screen showing the order life cycle

(paste your screen capture here)

Fig. 5.15 Order Management Screen

INSERT SCREENSHOT

Inventory screen with low-stock warnings

(paste your screen capture here)

Fig. 5.16 Inventory Management Screen

INSERT SCREENSHOT

Customer management screen

(paste your screen capture here)

Fig. 5.17 Customer Management Screen

INSERT SCREENSHOT

New customers report

(paste your screen capture here)

Fig. 5.18 New Customers Report

INSERT SCREENSHOT

Repeat customers report with order counts

(paste your screen capture here)

Fig. 5.19 Repeat Customers Report

INSERT SCREENSHOT

Loyalty configuration and ledger screen

(paste your screen capture here)

Fig. 5.20 Loyalty Management Screen

INSERT SCREENSHOT

Sales analytics screen

(paste your screen capture here)

Fig. 5.21 Sales Analytics Screen

INSERT SCREENSHOT

Demand prediction screen showing revenue, orders and baking quantities

(paste your screen capture here)

Fig. 5.22 Demand Prediction Screen

INSERT SCREENSHOT

Point-of-Sale counter billing screen

(paste your screen capture here)

Fig. 5.23 Point-of-Sale Counter Billing Screen

CHAPTER 6

TESTING

6.1 INTRODUCTION

Testing is the process of executing the software with the intention of finding defects. It was carried out continuously while the modules were being built and again after the application was assembled, so that a defect was discovered close to the point at which it was introduced. This chapter records the objectives of testing, the strategy followed at each level, the test cases executed and the results observed.

6.2 OBJECTIVES OF TESTING

- To verify that each module behaves in the way described by the functional requirements of Chapter 3.
- To discover and remove defects before the application is demonstrated.
- To confirm that the business rules, such as the minimum order value, the serviceable city, the stock check and the loyalty calculation, are enforced exactly.
- To confirm that the system refuses an unauthorised request and that a customer cannot reach an administrative function.
- To check that invalid input is rejected with a helpful message rather than being written to the database.

6.3 TESTING STRATEGY

6.3.1 Unit Testing

Each controller function was exercised on its own with representative inputs. The loyalty helper was checked for a range of order values, the coupon helper was checked for each supported code, and the prediction helper was checked for a weekday, a weekend and a festival date.

6.3.2 Integration Testing

The endpoints were called in the order in which a real user would call them: register, log in, list products, add to the cart, apply a coupon, place an order and read the order back. The aim was to confirm that the data written by one endpoint is exactly what the next endpoint reads, in particular that the stock deducted when an order is placed is the stock shown on the inventory screen.

6.3.3 System Testing

The complete application, with the front end, the back end and the database running together, was tested as a whole. The entire shopping journey was performed from a browser, and the counter journey was performed from the administration panel.

6.3.4 Validation Testing

Input validation was tested deliberately with wrong data: an empty cart, a subtotal below the minimum, a delivery city outside the serviceable area, an e-mail that is already registered, a coupon code that does not exist and a quantity larger than the available stock. In every case the system was expected to refuse the request and to explain why.

6.3.5 User Acceptance Testing

The application was demonstrated to a small group of users who were asked to complete two tasks without assistance: buy a cake online and bill a walk-in customer at the counter. Their observations about the wording of messages and the order of the fields were used to refine the interface.

6.4 TEST CASES

Table 6.1 lists the test cases executed against the system together with the expected and the observed result.

#	Test Case	Input	Expected Result	Observed
1	Customer registration	New name, e-mail, phone, password	Account created and a token returned	Pass
2	Duplicate registration	An e-mail already registered	Rejected with a conflict message	Pass
3	Valid login	Registered e-mail and correct password	Token returned with the correct role	Pass
4	Invalid login	Wrong password	Rejected with an invalid credentials message	Pass
5	Protected route without a token	A cart request with no header	Rejected with a no token message	Pass
6	Administrative route as a customer	A dashboard request with a customer token	Rejected with an access denied message	Pass
7	Browse the catalogue	Open the home page	Seven categories and the featured products are shown	Pass
8	Search for a product	The word cake	Only the matching products are returned	Pass
9	Natural-language query	Recommend a cake under six hundred rupees	A written answer and matching product cards	Pass
10	Add to cart	A product and a pack size	The line appears with the correct line total	Pass

#	Test Case	Input	Expected Result	Observed
11	Change the quantity	An increased quantity	The line total and the cart total are recalculated	Pass
12	Apply a valid coupon	The code SWEET10	A ten percent discount is applied	Pass
13	Apply an invalid coupon	An unknown code	Rejected with an invalid coupon message	Pass
14	Order below the minimum value	A subtotal below four hundred rupees	Rejected with the shortfall shown	Pass
15	Order to an unserviceable city	A city other than Chennai	Rejected with a not serviceable message	Pass
16	Place a valid order	A cart above the minimum and a Chennai address	Order created with a delivery OTP and a WhatsApp link	Pass
17	Loyalty points earned	An order of six hundred rupees	Thirty points credited to the customer	Pass
18	Loyalty points redeemed	A balance of fifty points and a larger bill	Fifty rupees deducted and the balance reduced	Pass
19	Repeat-customer discount	A customer with ten previous orders	A fifteen percent discount applied automatically	Pass
20	Counter sale with sufficient stock	Two items within the available stock	Sale recorded, stock deducted, points credited	Pass
21	Counter sale with insufficient stock	A quantity above the available stock	Rejected naming the item and the quantity available	Pass
22	Walk-in customer registration	A name and a phone number only	A customer record is created without a password	Pass
23	Demand prediction	A festival date	A forty-five percent surge applied and explained	Pass
24	Inventory update	A corrected stock quantity	The new quantity is shown and the warning clears	Pass
25	Order status change	Move an order to delivered	The status changes and the customer sees it	Pass

Table 6.1 Test cases and their results

6.5 RESULTS OF TESTING

All twenty-five test cases were executed and all of them passed. The defects found during the course of testing were of three kinds. First, a duplicate cart line was created when the same product and pack size was added twice; this was corrected by merging such lines and adding the quantities. Second, the stock of a product could fall below zero when two sales were made at the same moment; this was corrected by validating and deducting stock inside the same routine that creates the order. Third, the WhatsApp receipt link failed when the phone number

was stored with a country code or with spaces; this was corrected by stripping every non-digit character and using the last ten digits. After these corrections the application behaved as specified in every case that was tested.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 INTRODUCTION

This chapter evaluates the finished system against the objectives recorded in Chapter 2 and discusses what the results mean in practice for the bakery.

7.2 ACHIEVEMENT OF OBJECTIVES

Table 7.1 matches each objective to the way it was met.

Objective	How it was met	Status
Online catalogue	Seven categories, five products each, with pack sizes, prices, discounts and ratings	Achieved
Product discovery	Keyword search, category filters, sorting and an AI assistant that answers in ordinary language	Achieved
Secure authentication	bcrypt hashing, JSON Web Tokens and role-based access on every protected route	Achieved
Persistent cart and wishlist	One cart document per user restored on login; a single-call wishlist toggle	Achieved
Complete checkout	Address book, coupons, delivery slot, minimum value rule and serviceable-city rule	Achieved
Delivery confirmation	A four-digit OTP and a pre-filled WhatsApp receipt for every order	Achieved
Administration dashboard	Products, orders, inventory, customers, loyalty and reports in one panel	Achieved
Counter billing	Customer lookup, one-step walk-in registration, stock deduction and an order of type pos	Achieved
Loyalty programme	Five points per hundred rupees, one rupee per point on redemption, a full transaction ledger	Achieved
Repeat-customer rewards	Automatic discounts of five, ten and fifteen percent with new and repeat customer reports	Achieved
Demand prediction	EWMA smoothing over twenty days scaled by day-of-week and festival multipliers, with baking quantities	Achieved
Analytics	Revenue, average order value, best sellers, category performance and daily sales	Achieved
Maintainable structure	Separate models, controllers and routes; separate pages, components and contexts	Achieved

Table 7.1 Objectives and their achievement

7.3 OBSERVATIONS

- Separating the interface from the API made the work considerably easier. When the loyalty rule was changed from four to five points per hundred rupees, only one controller had to be edited; no screen needed to be touched.
- The prediction module is most useful to the bakery not because the number it produces is exact, but because it is consistent and because it explains itself. An owner who is told that Thursday carries a twenty percent uplift can agree or disagree from experience, and the multiplier can be corrected in one place.
- Running the counter channel and the online channel through the same order model turned out to be the most valuable design decision in the project. It gives the owner one revenue figure, one stock figure and one customer list, which is precisely what the manual system could never provide.
- The JSON fallback store kept the application usable during development whenever the hosted cluster could not be reached. It is not a substitute for a real database in production, but it made the project far easier to demonstrate.
- Validation is what the user actually notices. A clear message explaining that forty rupees more are needed to reach the minimum order value is far better than a silent failure, and adding such messages removed most of the confusion seen during acceptance testing.

7.4 DISCUSSION

The system meets every objective that was set for it. Its main strength is that it unifies two channels of sale that in a traditional bakery are entirely separate, and that it turns every transaction into a record that can be used later for forecasting and for rewarding the customer. Its main weakness is that it does not yet take payment: the method is recorded, but no money moves. That is the first thing that would have to be addressed before the system could be used commercially. The second is that the prediction depends on the quality and the quantity of the history recorded; a bakery that adopts the system will need a few weeks of ordinary trading before the forecast can be trusted.

CHAPTER 8

CONCLUSION AND FUTURE ENHANCEMENTS

8.1 CONCLUSION

Creamy Bites was built to solve a set of problems that a neighbourhood bakery faces every day: no catalogue on the Internet, orders written by hand, no knowledge of stock until it is too late, no record of who the customers are, slow counter billing and no way of guessing how much to bake. The system that was developed answers each of these with a working feature rather than with a proposal.

A customer can browse, search, ask the assistant in ordinary language, keep a cart and a wishlist, save addresses, apply a coupon and pay on delivery, and receives an OTP and a WhatsApp receipt for every order. An administrator can manage the catalogue, move orders through their life cycle, watch stock and correct it, study revenue and best sellers, and understand the difference between new and repeat customers. A counter assistant can serve a walk-in customer in a few seconds and still record the sale, deduct the stock and update the loyalty balance. The loyalty engine rewards a customer at five points per hundred rupees and accepts a point as one rupee, and the repeat customer engine adds a discount of five, ten or fifteen percent as the customer returns. The prediction engine looks twenty days back, smooths what it finds and tells the owner what tomorrow is likely to bring and how much to bake.

Every one of the twenty-five test cases that were executed passed. The objectives listed in Chapter 2 were all met, and the limitations that remain are recorded openly in Chapter 2 and discussed in Chapter 7 rather than being hidden. The project therefore stands as a complete and honest piece of work, and it gave a thorough grounding in building a modern three-tier web application, in designing a REST interface, in modelling data for a real business and in testing a system until it can be trusted.

8.2 FUTURE ENHANCEMENTS

- **Payment gateway integration.** Connecting a UPI and card gateway would allow the system to capture payment online, record the transaction reference against the order and mark the payment status as paid automatically.
- **Real-time delivery tracking.** Showing the position of the delivery agent on a map, with an estimated time of arrival, would improve the experience between the order being packed and the order being delivered.
- **Native mobile applications.** Because all business functions are already exposed through the REST API, an Android and an iOS client could be built without any change

to the server.

- **Automatic purchase ordering.** The prediction module could be extended to compare the recommended baking quantity with the stock of raw material and to draft a purchase order for the supplier.
- **Richer forecasting.** The present multiplier table is deliberately simple. It could be replaced by a model that also considers the weather, local events and the day of the lunar calendar, and that learns its own multipliers from the errors it has made.
- **Notifications.** Integrating an e-mail and an SMS service provider would let the system send the receipt, the dispatch notice and the delivery confirmation without depending on the customer opening a link.
- **Multi-language interface and multiple stores.** Supporting Tamil and Hindi would widen the customer base, and supporting more than one outlet would let a small chain share one catalogue and one customer list.
- **Advanced reporting.** A profitability report that subtracts the cost of ingredients and the value of the stock written off at the end of the day would tell the owner not only what sold, but what actually earned.

REFERENCES

[1] E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA, USA: Addison-Wesley, 1994.

[2] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ, USA: Prentice Hall, 2008.

[3] I. Sommerville, *Software Engineering*, 10th ed. Harlow, United Kingdom: Pearson Education, 2015.

[4] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill Education, 2014.

[5] K. Beck and C. Andres, *Extreme Programming Explained: Embrace Change*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2004.

[6] React Documentation, "React 18 - Getting Started." [Online]. Available: https://react.dev

[7] Vite Documentation, "Getting Started with Vite." [Online]. Available: https://vitejs.dev

[8] Express.js Documentation, "Express - Node.js web application framework." [Online]. Available: https://expressjs.com

[9] MongoDB Documentation, "MongoDB Manual." [Online]. Available: https://www.mongodb.com/docs

[10] Mongoose Documentation, "Mongoose ODM." [Online]. Available: https://mongoosejs.com/docs

[11] Auth0, "JSON Web Token Introduction." [Online]. Available: https://jwt.io

[12] MDN Web Components and Browser APIs, "Using the Fetch API and HTTP access control." [Online]. Available: https://developer.mozilla.org

[13] N. Ford, "Exponential Smoothing and its Variants," *International Journal of Forecasting*, vol. 22, no. 4, pp. 637-646, 2006.

[14] A. Turban, J. Outland, D. King, J. Lee, T.-P. Liang and D. Turban, *Electronic Commerce: A Managerial and Social Networks Perspective*, 9th ed. Cham, Switzerland: Springer, 2018.